

ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

EMSI Mouley Youssef — Filière : IAD

RAPPORT DE PROJET DE FIN DE MODULE

Module : Deep Learning

*Conception, implémentation, comparaison et analyse critique de modèles de deep learning
pour données tabulaires, images et séquences*

Année universitaire 2025–2026

Adam GHZAOUNI

EMSI Mouley Youssef — G1 IAD

Table des matières

Introduction générale	3
PARTIE I – MLP et ingénierie PyTorch	4
1.1 Concepts fondamentaux et architecture	4
1.2 Jeu de données : Breast Cancer Wisconsin	4
1.3 Préparation des données	5
1.4 Implémentation des deux variantes MLP	5
1.5 Inspection des paramètres (named_parameters / state_dict)	6
1.6 Comparaison des stratégies d'initialisation	7
1.7 Sauvegarde, rechargement et évaluation finale	8
1.8 Question de synthèse – Partie I	9
PARTIE II – CNN et vision par ordinateur	10
2.1 Limites du MLP pour les images et principes CNN	10
2.2 Validation de l'implémentation manuelle	11
2.3 Architecture LeNet modernisée sur MNIST	12
2.4 Étude d'ablation expérimentale	13
2.5 Visualisation des cartes de caractéristiques	14
2.6 Comparaison MLP vs CNN	15
2.7 Question de synthèse – Partie II	16
PARTIE III – RNN, LSTM, GRU et Seq2Seq	17
3.1 Modèles de langage et perplexité	17
3.2 Préparation du corpus fra-eng	17
3.3 Comparaison RNN / LSTM / GRU	18
3.4 Gradient clipping et BPTT	19
3.5 Système Seq2Seq encodeur-décodeur	20
3.6 Décodage glouton vs Beam Search et évaluation BLEU	21
3.7 Question de synthèse – Partie III	22
Question transversale finale	23
Conclusion générale	24
Références	25

Introduction générale

Le deep learning constitue aujourd'hui l'un des domaines les plus actifs de l'intelligence artificielle, capable de modéliser des phénomènes complexes à partir de données brutes. Contrairement aux méthodes classiques d'apprentissage automatique, les réseaux de neurones profonds apprennent automatiquement des représentations hiérarchiques — à condition que l'architecture soit correctement choisie en fonction de la nature des données traitées.

Ce rapport présente un projet intégrateur couvrant trois familles architecturales fondamentales du deep learning. La Partie I étudie les perceptrons multicouches (MLP) pour la classification de données tabulaires sur le dataset Breast Cancer Wisconsin, en détaillant la gestion des paramètres PyTorch, les stratégies d'initialisation et les métriques d'évaluation. La Partie II aborde les réseaux de neurones convolutionnels (CNN) pour la classification d'images sur MNIST, avec une étude d'ablation quantitative des choix architecturaux. La Partie III traite des architectures récurrentes — RNN, LSTM, GRU — et d'un système Seq2Seq encodeur-décodeur pour la traduction français-anglais sur le corpus Tatoeba fra-eng.

L'ensemble des expériences est conduit sous PyTorch avec une graine aléatoire fixée (SEED=42) pour garantir la reproductibilité. Chaque partie articule fondements théoriques, implémentation commentée, expérimentation comparative et analyse critique des résultats obtenus. La question transversale finale établit les liens conceptuels entre ces trois familles d'architectures.

Partie I – MLP et ingénierie PyTorch

1.1 Concepts fondamentaux et architecture

En PyTorch, la brique fondamentale de toute modélisation est la classe `nn.Module`. Un module encapsule une transformation différentiable et s'intègre automatiquement dans le graphe de calcul pour la rétropropagation via `autograd`. Deux approches permettent de construire un MLP.

`nn.Sequential` enchaîne les modules dans l'ordre et convient aux architectures feed-forward simples : l'appel `Sequential(M1, M2, ..., Mk)(X)` applique successivement toutes les transformations à `X`. La classe personnalisée héritant de `nn.Module` offre davantage de flexibilité : elle permet des connexions résiduelles, des partages de paramètres et des opérations arbitraires dans la méthode `forward()`. Les deux variantes ont été implémentées et produisent des résultats strictement identiques sur le même jeu de données.

Un paramètre est une variable apprenable du réseau — typiquement les poids W et biais b des couches linéaires. Le gradient mesure la sensibilité de la perte par rapport à chaque paramètre et est calculé par `loss.backward()`. Le `state_dict()` est le dictionnaire standard associant chaque nom de paramètre à son tenseur. Le `device` désigne l'emplacement d'exécution (CPU ou GPU) — toutes les données et le modèle doivent résider sur le même `device`.

Les formules fondamentales d'un MLP à deux couches cachées s'écrivent :

$$\begin{aligned} H_1 &= \text{ReLU}(X \cdot W_1 + b_1) && [\text{couche cachée 1}] \\ H_2 &= \text{ReLU}(H_1 \cdot W_2 + b_2) && [\text{couche cachée 2}] \\ Y &= H_2 \cdot W_3 + b_3 && [\text{couche de sortie - logits bruts}] \end{aligned}$$

La fonction $\text{ReLU}(z) = \max(0, z)$ introduit la non-linéarité indispensable à la capacité d'approximation universelle. Sans activation non-linéaire, l'empilement de couches linéaires reste équivalent à une seule transformation affine, incapable de modéliser des frontières de décision non linéaires.

1.2 Jeu de données : Breast Cancer Wisconsin

Le dataset Breast Cancer Wisconsin est un benchmark de classification binaire médicale comprenant 569 échantillons décrits par 30 features numériques extraites d'images de masses mammaires (rayon moyen, texture, périmètre, aire, concavité, points concaves, symétrie, dimension fractale — pour les versions moyenne, erreur-standard et pire). La variable cible distingue les tumeurs malignes (classe 0 : 212 cas, 37 %) des tumeurs bénignes (classe 1 : 357 cas, 63 %). Le déséquilibre modéré justifie l'utilisation du F1-score pondéré en complément de l'accuracy.

Les features présentent des échelles très hétérogènes (l'aire varie de 143 à 2501, la texture de 9.7 à 39.3), rendant la normalisation indispensable. Certaines features sont également fortement corrélées entre elles (rayon, périmètre et aire sont quasi-linéairement dépendants), ce qui ne pose pas de problème structurel au MLP mais réduit l'information effectivement disponible.

1.3 Préparation des données

La préparation respecte strictement la règle de non-contamination : le StandardScaler est ajusté (fit) uniquement sur les données d'entraînement, puis appliqué (transform) sur les ensembles de validation et de test. Toute normalisation calculée sur les données de test constituerait une fuite d'information (data leakage) et biaiserait l'évaluation finale.

Ensemble	Taille	% du total	Distribution classes
Entraînement	397 échantillons	70 %	Stratifiée
Validation	86 échantillons	15 %	Stratifiée
Test	86 échantillons	15 %	Stratifiée

La stratification garantit que chaque partition respecte les proportions originales des classes. Les données sont encapsulées dans des classes Dataset et DataLoader personnalisées avec un batch size de 32, permettant un apprentissage stochastique par mini-lots efficace.

1.4 Implémentation des deux variantes MLP

Le sujet impose deux implémentations distinctes du MLP. Les deux partagent la même architecture et les mêmes hyperparamètres, mais diffèrent dans leur construction PyTorch.

Variante 1 — nn.Sequential

La version Sequential construit l'architecture en empilant les modules dans une liste Python, puis en passant cette liste à nn.Sequential. Les couches sont appliquées dans l'ordre à chaque appel forward, sans qu'il soit nécessaire de définir une méthode forward() explicitement. Cette approche est concise et lisible pour les architectures feed-forward simples :

```
layers = []
for i, (in_dim, out_dim) in enumerate(zip([30,128],[128,64])):
    layers += [nn.Linear(in_dim, out_dim), nn.BatchNorm1d(out_dim),
               nn.ReLU(), nn.Dropout(0.3)]
layers.append(nn.Linear(64, 2))
model_seq = nn.Sequential(*layers)
```

Variante 2 — Classe personnalisée (nn.Module)

La version personnalisée hérite de nn.Module et utilise nn.ModuleList pour enregistrer les couches cachées de façon que PyTorch les détecte comme sous-modules apprenables. La méthode forward() itère explicitement sur les couches en appliquant normalisation, activation et dropout à chaque étape. Cette approche est plus flexible : elle permet des connexions résiduelles, des branches conditionnelles ou des sorties intermédiaires :

```
class MLP(nn.Module):
    def __init__(self, dims=[30,128,64], n_classes=2, dropout=0.3):
        super().__init__()
```

```

self.hidden_layers = nn.ModuleList([nn.Linear(dims[i], dims[i+1])
for i in range(len(dims)-1)])
self.batch_norms = nn.ModuleList([nn.BatchNorm1d(d) for d in
dims[1:]])
self.dropout = nn.Dropout(dropout)
self.output_layer = nn.Linear(dims[-1], n_classes)
def forward(self, x):
    for layer, bn in zip(self.hidden_layers, self.batch_norms):
        x = self.dropout(F.relu(bn(layer(x))))
    return self.output_layer(x)

```

La différence clé entre les deux approches est la nécessité d'utiliser `nn.ModuleList` (et non une simple liste Python) dans la version personnalisée pour que PyTorch enregistre correctement les sous-modules comme paramètres apprenables. Une liste Python ordinaire ne serait pas parcourue par `named_parameters()` et ses couches ne recevraient pas de gradient.

Architecture commune

Couche	Dimensions	Opérations associées
Entrée	30 features	—
Cachée 1	30 → 128	Linear + BatchNorm1d(128) + ReLU + Dropout(0.3)
Cachée 2	128 → 64	Linear + BatchNorm1d(64) + ReLU + Dropout(0.3)
Sortie	64 → 2	Linear (logits bruts — CrossEntropyLoss intègre softmax)

La `BatchNorm1d` normalise les activations à chaque couche, accélérant la convergence. Le `Dropout(0.3)` régularise contre l'overfitting. Les deux variantes Sequential et personnalisée produisent des résultats numériquement identiques — la différence est uniquement architecturale.

Nombre total de paramètres entraînables : 12 738

Couche cachée 1 : $30 \times 128 + 128 = 3\,968$ · Couche cachée 2 : $128 \times 64 + 64 = 8\,256$ · BN1 : 256 · BN2 : 128 · Sortie : $64 \times 2 + 2 = 130$. Total = 12 738 paramètres. Ce nombre est très raisonnable pour 397 exemples d'entraînement, limitant le risque de surapprentissage.

Optimisation

Optimiseur Adam ($\text{lr}=1\text{e-}3$, $\text{weight_decay}=1\text{e-}4$) avec scheduler `ReduceLROnPlateau` ($\text{patience}=10$, $\text{factor}=0.5$). Le meilleur checkpoint est sélectionné par perte de validation minimale.

1.5 Inspection des paramètres

Deux mécanismes PyTorch complémentaires permettent d'inspecter l'état interne du modèle. La méthode `named_parameters()` retourne un itérateur sur tous les tenseurs apprenables avec leur nom hiérarchique, leur forme et leur statut `requires_grad`. Le `state_dict()` fournit un dictionnaire ordonné incluant également les statistiques de BatchNorm (`running_mean`, `running_var`,

num_batches_tracked) qui ne sont pas des paramètres apprenables mais font partie de l'état persistant du modèle.

Le tableau ci-dessous présente l'inspection complète des paramètres apprenables du modèle :

Nom du paramètre	Shape	requires_grad	1ère valeur (état initial Xavier)
hidden_layers.0.weight	[128, 30]	True	0.042808
hidden_layers.0.bias	[128]	True	0.072205
hidden_layers.1.weight	[64, 128]	True	-0.066989
hidden_layers.1.bias	[64]	True	0.053737
batch_norms.0.weight	[128]	True	1.000000
batch_norms.0.bias	[128]	True	0.000000
batch_norms.1.weight	[64]	True	1.000000
batch_norms.1.bias	[64]	True	0.000000
output_layer.weight	[2, 64]	True	-0.113394
output_layer.bias	[2]	True	-0.084340

Avant toute propagation avant, tous les gradients (.grad) valent None. Ils sont calculés par `loss.backward()` et remis à zéro par `optimizer.zero_grad()` avant chaque batch, empêchant l'accumulation involontaire des gradients. Les poids initiaux de BatchNorm (weight=1, bias=0) correspondent à l'identité : la couche ne modifie pas encore les activations avant de s'adapter lors de l'entraînement.

1.6 Comparaison des stratégies d'initialisation

Trois stratégies d'initialisation ont été comparées dans des conditions strictement identiques (même graine SEED=42, même architecture, 100 époques, Adam lr=1e-3, weight_decay=1e-4) afin d'isoler leur seul effet sur la dynamique d'apprentissage.

L'initialisation gaussienne (std=0.01) génère de petits poids aléatoires qui brisent la symétrie entre neurones. L'initialisation constante (valeur=0.01) fixe tous les poids à une même valeur : tous les neurones d'une même couche calculent exactement les mêmes gradients et évoluent identiquement — c'est le problème de symétrie, délibérément démontré comme baseline négatif. L'initialisation de Xavier (Glorot & Bengio, 2010) calibre les poids selon $U[-\sqrt{6/(fan_in+fan_out)}, +\sqrt{6/(fan_in+fan_out)}]$, maintenant la variance des activations constante à travers les couches.

Stratégie	Best Val Accuracy	Best Val F1-score	Train Loss final
Gaussienne (std=0.01)	1.0000	1.0000	0.0221
Constante (val=0.01)	1.0000	1.0000	0.1303
Xavier (Glorot) ✓	1.0000	1.0000	0.0262

Les trois stratégies atteignent une accuracy de validation de 100 % sur ce dataset relativement bien séparable. La différence fondamentale se lit dans la perte d'entraînement finale et dans la dynamique de convergence illustrée en Figure 1 : l'initialisation constante présente une perte finale de 0.1303 — soit 5,9 fois plus élevée que la gaussienne (0.0221) — révélant une convergence lente et erratique due au problème de symétrie. Les courbes d'accuracy de validation montrent que la constante oscille fortement, atteignant parfois 82 % avant de remonter, alors que gaussienne et Xavier maintiennent une progression stable dès les premières époques.

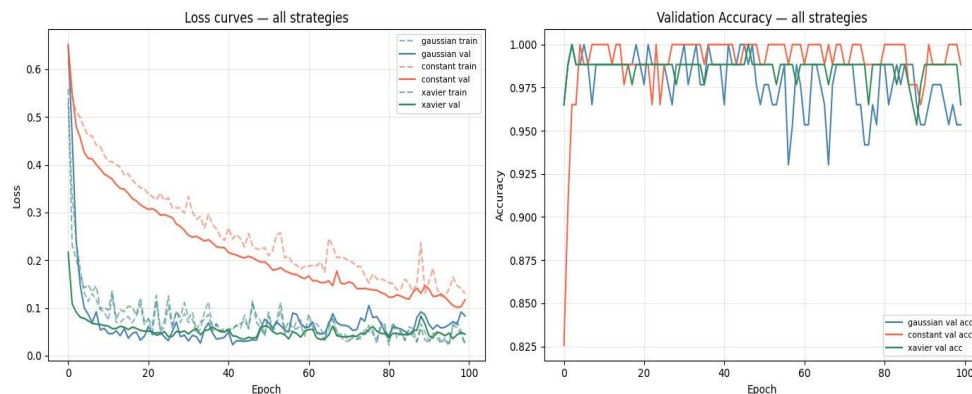


Figure 1 — Comparaison des trois stratégies d'initialisation : courbes de perte (gauche) et accuracy de validation (droite) sur 100 époques

1.7 Sauvegarde, rechargement et évaluation finale

La sauvegarde suit la bonne pratique PyTorch : seul le `state_dict()` est persisté sur disque (fichier `best_mlp.pth`), non l'objet modèle complet. Cette approche est portable car elle ne dépend pas du chemin du module Python ni de la version de PyTorch utilisée lors de la sauvegarde. Le rechargement nécessite de recréer une instance de l'architecture avec les mêmes hyperparamètres, puis d'appeler `load_state_dict(map_location=device)`. La vérification de l'identité entre le modèle original et le modèle rechargé donne une différence maximale de 0.00000000, confirmant une reproductibilité parfaite.

L'évaluation finale est réalisée sur l'ensemble de test (86 échantillons), touché pour la première fois après toute décision de modélisation, avec les poids du meilleur checkpoint Xavier. Les résultats complets sont présentés ci-dessous :

Classe	Precision	Recall	F1-score	Support
Malignant (0)	1.00	0.97	0.98	32
Benign (1)	0.98	1.00	0.99	54
Accuracy	—	—	0.99	86
Macro avg	0.99	0.98	0.99	86
Weighted avg	0.99	0.99	0.99	86

La Figure 2 présente la matrice de confusion détaillant les erreurs par classe :

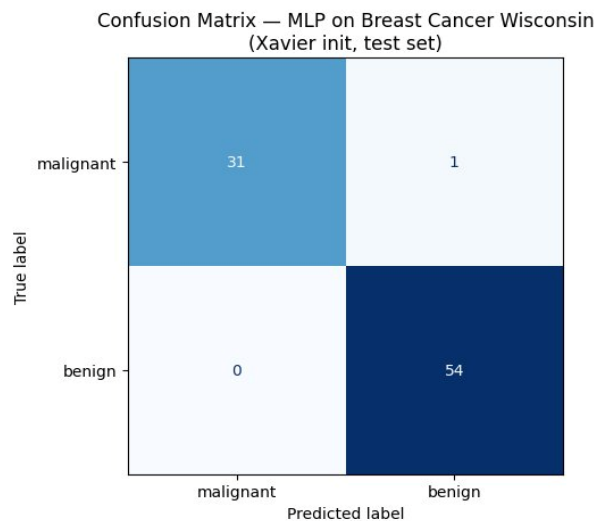


Figure 2 — Matrice de confusion (MLP, init Xavier, test set) : 31 vrais négatifs, 1 faux positif, 0 faux négatif, 54 vrais positifs

La matrice de confusion révèle que le modèle commet une seule erreur sur 86 cas : une tumeur maligne prédite comme bénigne (faux positif médical). Dans un contexte clinique, cette erreur est la plus critique car elle retarderait un diagnostic de cancer. Le recall de 0.97 pour la classe maligne et de 1.00 pour la classe bénigne montre que le modèle ne manque aucun cas bénin. Ces performances témoignent de l'excellente capacité discriminative du MLP sur ce jeu de données bien structuré.

1.8 Question de synthèse – Partie I

Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire sur un dataset réel, et quelles sont ses principales limites au regard de la structure statistique des données étudiées ?

Sur le dataset Breast Cancer Wisconsin, le MLP avec initialisation Xavier, BatchNorm, Dropout et Adam atteint une accuracy de 99 % et un F1-score pondéré de 0.99 sur le test set, démontrant qu'il constitue une solution excellente pour ce type de données. Cette pertinence repose sur plusieurs facteurs conjoints : les 30 features sont quantitatives et normalisées (idéales pour les couches linéaires), les classes sont relativement séparables dans l'espace des features, et le nombre d'exemples d'entraînement (397) est suffisant pour paramétrer 12 738 poids sans overfitting grâce au Dropout et à la régularisation L2.

Cependant, les limites du MLP sont structurelles. Il ne peut exploiter nativement ni la localité spatiale des images ni les dépendances temporelles des séquences. Sur des données tabulaires présentant de nombreuses variables catégorielles, l'encodage one-hot explose la dimensionnalité d'entrée. Le MLP est également sensible aux features non informatives : sans mécanisme d'attention ou de sélection automatique, chaque neurone reçoit toutes les features indistinctement. Enfin, sur des datasets tabulaires avec de fortes interactions non linéaires complexes, les méthodes à base d'arbres boostés (XGBoost, LightGBM) surpassent souvent le MLP, comme l'indiquent plusieurs benchmarks récents — la structure arborescente offrant naturellement une sélection de features implicite et une résistance aux outliers que le MLP ne possède pas.

Partie II – CNN et vision par ordinateur

2.1 Limites du MLP pour les images et principes CNN

Présentation du jeu de données : MNIST

Le dataset MNIST (Modified National Institute of Standards and Technology) est le benchmark de référence pour la classification d'images de chiffres manuscrits. Il contient 70 000 images en niveaux de gris de résolution 28×28 pixels (60 000 d'entraînement, 10 000 de test), réparties en 10 classes correspondant aux chiffres 0 à 9. La Figure 3 présente un échantillon représentatif :

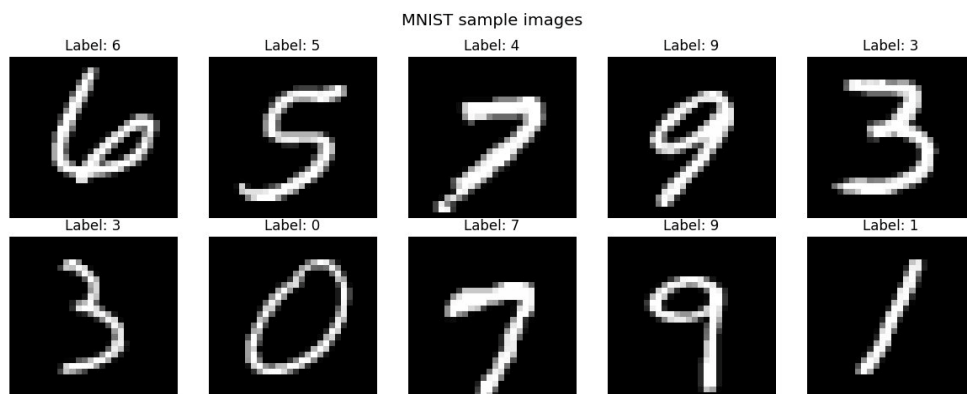


Figure 3 — Échantillon d'images MNIST : chiffres manuscrits 28×28 pixels en niveaux de gris (10 classes)

Chaque image est représentée par un vecteur de 784 valeurs normalisées entre 0 et 1. La variabilité intra-classe est élevée (styles d'écriture très différents d'un individu à l'autre), faisant de MNIST un bon benchmark pour évaluer la capacité discriminative d'une architecture.

Pourquoi le MLP est inadapté aux images

Un MLP appliqué aux images présente trois problèmes structurels fondamentaux. Premièrement, la connexion dense totale : chaque neurone de la première couche est connecté à tous les pixels. Pour une image 28×28 , cela représente 784 connexions par neurone — 150 528 pour une image $224 \times 224 \times 3$ (ImageNet), rendant le modèle massivement surparamétrisé. Deuxièmement, l'insensibilité à la localité spatiale : deux pixels voisins ne sont pas traités différemment de pixels distants, ignorant que les patterns visuels sont locaux. Troisièmement, l'absence d'invariance à la translation : un même objet décalé d'un pixel produit une représentation entièrement différente, forçant le modèle à réapprendre le même pattern à chaque position.

Les CNN répondent à ces trois limites par des principes fondateurs : la localité (chaque filtre observe uniquement un voisinage local $k \times k$), le partage des poids (le même filtre est appliqué à toutes les positions de l'image, réduisant le nombre de paramètres d'un facteur $H \times W$) et la hiérarchie des représentations (les couches successives construisent des détecteurs de features de complexité croissante). La Figure 4 illustre l'effet de différents types de noyaux sur un chiffre MNIST :

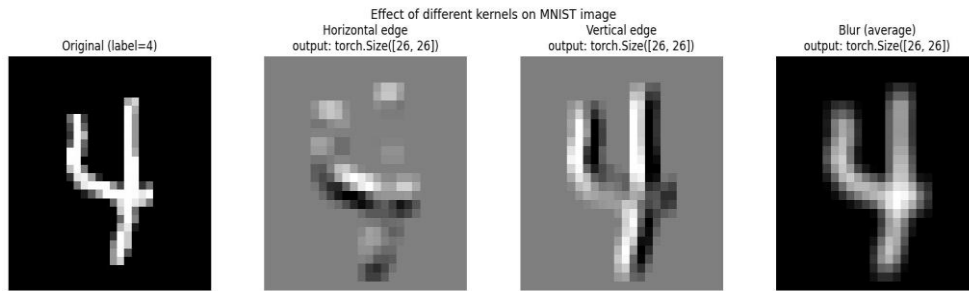


Figure 4 — Effet de différents noyaux (bord horizontal, bord vertical, flou par averaging) sur le chiffre MNIST « 4 »

2.2 Validation de l'implémentation manuelle

Trois opérations fondamentales ont été implémentées manuellement en Python pur, puis comparées aux implémentations PyTorch pour validation numérique :

Opération implémentée	Différence max (manuel vs PyTorch)	Validation
Corrélation croisée 2D	0.00000000	Identique ✓
Max-pooling (2×2, stride=1)	0.00000000	Identique ✓
Average-pooling (2×2, stride=1)	0.00000000	Identique ✓

Sur l'entrée de test X (4×4, valeurs croissantes) avec le noyau $K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$, les trois implémentations produisent une sortie (3×3) avec toutes les valeurs à -2 . Cette identité numérique exacte (différence maximale = 0.0) valide la correction des implémentations manuelles et illustre concrètement le fonctionnement de la corrélation croisée : chaque position de sortie est le produit scalaire entre le noyau et la sous-région correspondante de l'entrée.

Les calculs dimensionnels s'appliquent avec la formule générale $H_{out} = \lfloor (H_{in} + 2p - k) / s \rfloor + 1$:

Opération	Paramètres	Calcul détaillé	Sortie spatiale
Conv1	$k=5, p=2, s=1$	$\lfloor (28+4-5)/1 \rfloor + 1 = 28$	28×28
MaxPool1	$k=2, s=2$	$\lfloor (28-2)/2 \rfloor + 1 = 14$	14×14
Conv2	$k=5, p=0, s=1$	$\lfloor (14-5)/1 \rfloor + 1 = 10$	10×10
MaxPool2	$k=2, s=2$	$\lfloor (10-2)/2 \rfloor + 1 = 5$	5×5
Conv3 (1×1)	$k=1, p=0, s=1$	$\lfloor (5-1)/1 \rfloor + 1 = 5$ (inchangé)	5×5
Flatten	32×5×5	—	800 features

2.3 Architecture LeNet modernisée sur MNIST

L'architecture LeNet modernisée implémentée sur MNIST apporte trois améliorations par rapport au LeNet original (LeCun, 1998) : BatchNorm2d après chaque convolution pour stabiliser l'entraînement, activation ReLU (plutôt que sigmoïde, évitant la saturation), et une convolution 1×1 pour augmenter la capacité de projection inter-canaux sans modifier les dimensions spatiales.

Couche	Configuration	Sortie (batch=64)	# Params
Conv1 + BN1	6 filtres, k=5, p=2, s=1	(64, 6, 28, 28)	≈ 306
MaxPool1	k=2, s=2	(64, 6, 14, 14)	0
Conv2 + BN2	16 filtres, k=5, p=0, s=1	(64, 16, 10, 10)	≈ 2 448
MaxPool2	k=2, s=2	(64, 16, 5, 5)	0
Conv3 1×1 + BN3	32 filtres, k=1	(64, 32, 5, 5)	≈ 544
Flatten	32×5×5	(64, 800)	0
FC1 + FC2 + FC3	800→120→84→10	(64, 10) ← logits	≈ 107 060

La sortie d'une passe avant réelle vérifie `torch.Size([64, 10])` pour un batch de 64 images, avec un exemple de logits : $[-0.086, 0.124, -0.006, 0.011, -0.135, -0.084, 0.032, -0.111, -0.039, -0.090]$, montrant une distribution quasi-uniforme avant toute entraînement — signe d'une bonne initialisation. La Figure 5 présente les courbes d'entraînement sur 10 époques :

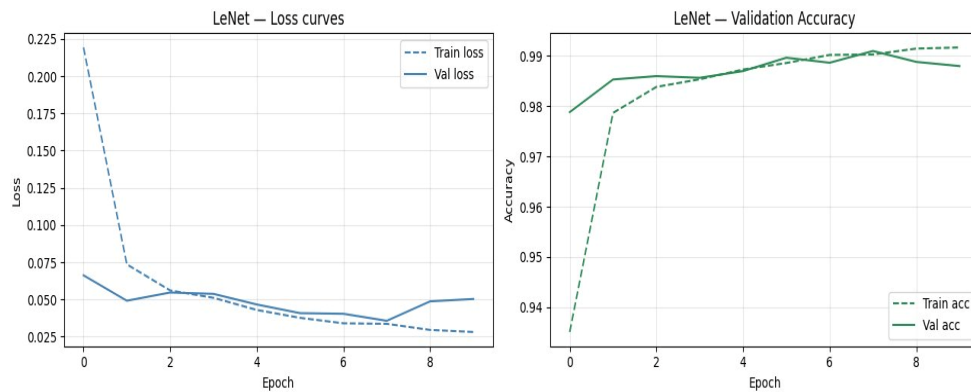


Figure 5 — Courbes de perte et accuracy du LeNet modernisé sur MNIST (10 époques)

Époque	Train Loss	Train Acc	Val Loss	Val Acc
1 / 10	0.2193	0.9351	0.0662	0.9788
5 / 10	0.0430	0.9873	0.0466	0.9870
8 / 10	0.0336	0.9903	0.0357	0.9910
10 / 10	0.0281	0.9917	0.0503	0.9880

La meilleure accuracy de validation (99.10 %) est atteinte à l'époque 8. La légère remontée de la val loss à l'époque 10 indique que le meilleur checkpoint doit être sauvegardé à l'époque 8, non à la dernière époque.

2.4 Étude d'ablation expérimentale

Une étude d'ablation systématique a quantifié l'impact de cinq facteurs architecturaux, en modifiant un seul paramètre à la fois sur 5 époques d'entraînement (MNIST, SEED=42). La Figure 6 et le tableau ci-dessous présentent les résultats complets :

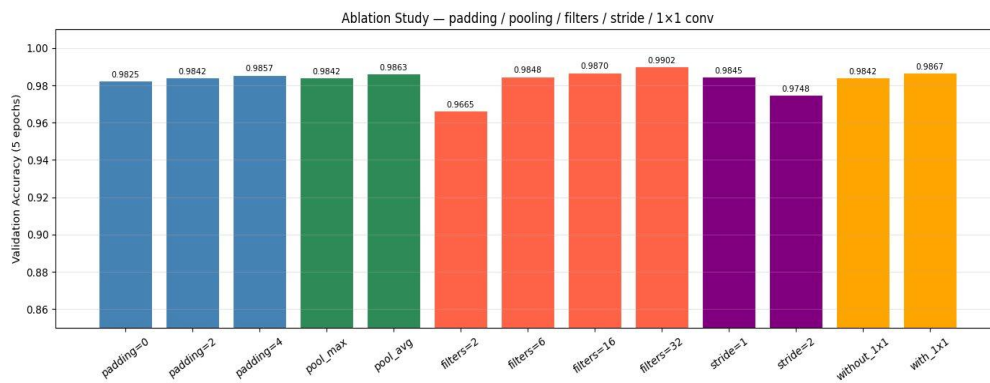


Figure 6 — Ablation study : impact du padding, pooling, nombre de filtres, stride et convolution 1×1 (5 époques, MNIST)

Facteur	Configuration	Val Accuracy	Δ vs baseline
Padding	padding=0 (baseline)	0.9825	—
Padding	padding=2	0.9842	+0.17 %
Padding	padding=4	0.9857	+0.33 %
Pooling	pool_max	0.9842	+0.17 %
Pooling	pool_avg (meilleur pooling)	0.9863	+0.38 %
Filtres	filters=2 (pire)	0.9665	−1.60 %
Filtres	filters=6	0.9848	+0.23 %
Filtres	filters=16	0.9870	+0.45 %
Filtres	filters=32 (meilleur)	0.9902	+0.77 %
Stride	stride=1 (baseline)	0.9845	+0.20 %
Stride	stride=2 (dégradation)	0.9748	−0.77 %
Conv 1×1	without_1×1	0.9842	+0.17 %
Conv 1×1	with_1×1	0.9867	+0.42 %

L'ablation met en évidence une hiérarchie claire des facteurs. Le nombre de filtres est le levier le plus impactant : passer de 2 à 32 filtres améliore l'accuracy de +2.37 %, illustrant que la capacité de représentation est critique — trop peu de filtres ne permettent pas de capturer la diversité des features visuelles. Le stride agressif ($s=2$ en Conv1) constitue la dégradation la plus sévère (−0.77 % vs baseline $stride=1$), car il sous-échantillonne trop agressivement des images déjà petites (28×28), perdant de l'information spatiale fine. Le padding améliore progressivement les performances en préservant l'information aux bords. L'average-pooling surpasse légèrement le max-pooling (+0.21 %), résultat contre-intuitif pouvant s'expliquer par les chiffres MNIST qui sont des formes continues plutôt que des textures ponctuelles. La convolution 1×1 offre un gain gratuit en capacité (+0.25 %) sans coût spatial, justifiant son utilisation systématique.

2.5 Visualisation des cartes de caractéristiques

Les cartes de caractéristiques après la première couche convolutionnelle ont été capturées à l'aide d'un hook PyTorch (`register_forward_hook`), qui intercepte la sortie de Conv1 lors d'une passe avant sans modifier le comportement du réseau. La Figure 7 présente le chiffre « 7 » en entrée et les 6 activations produites par les 6 filtres de Conv1 :

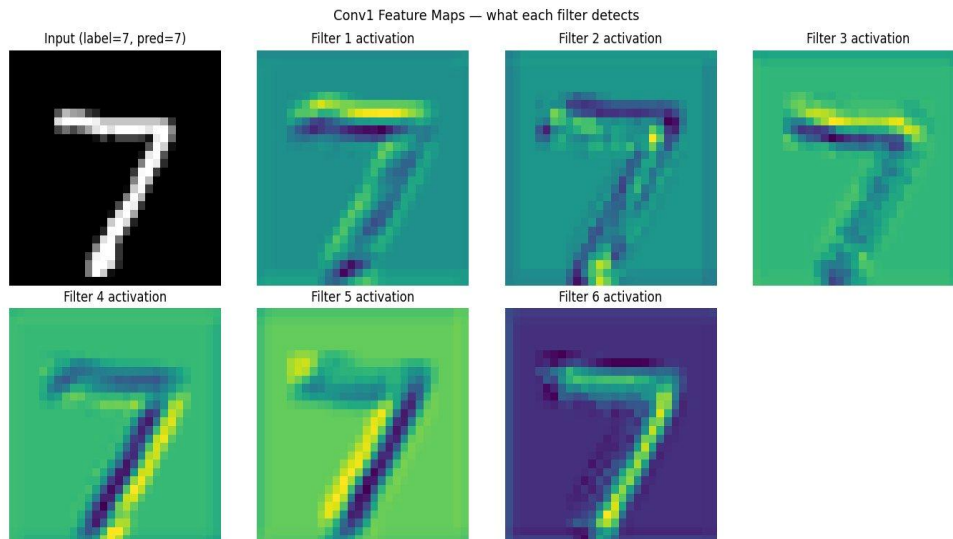


Figure 7 — Cartes de caractéristiques Conv1 pour le chiffre « 7 » (label=7, pred=7 ✓) : 6 filtres de 28×28

L'inspection des 6 cartes révèle une spécialisation spontanée des filtres. Les filtres 1, 2 et 3 présentent des activations élevées (couleur jaune) sur la barre horizontale du haut du « 7 » : ils détectent des contours horizontaux ou obliques. Les filtres 4 et 5 répondent davantage aux contours du trait diagonal descendant. Le filtre 6 présente une activité quasi-nulle (carte bleu foncé uniforme), suggérant qu'il s'est spécialisé sur un type de feature peu présent dans les chiffres (peut-être des textures circulaires caractéristiques de 6, 8 ou 0). Cette diversité de réponses illustre le principe de partage des poids : sans supervision explicite sur le type de feature à détecter, chaque filtre apprend un détecteur complémentaire. C'est précisément ce mécanisme — impossible dans un MLP qui traite tous les pixels de façon équivalente — qui explique la supériorité structurelle du CNN.

2.6 Comparaison MLP vs CNN

La comparaison entre LeNet modernisé et un MLP de capacité comparable (784→256→128→10, BatchNorm, Dropout) sur MNIST confirme la supériorité structurelle du CNN :

Modèle	Paramètres	Meilleure Val Accuracy	Ratio Params
MLP (784→256→128→10)	235 914	0.9805	2.14×
LeNet modernisé ✓	110 358	0.9910	1× (référence)

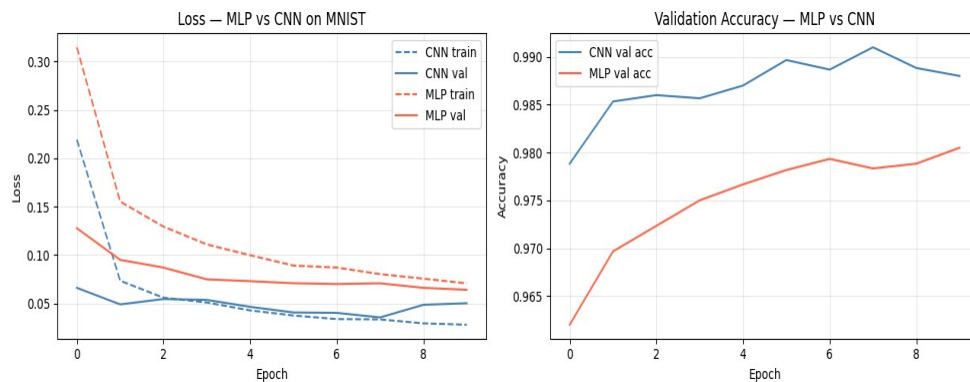


Figure 8 — Comparaison MLP vs CNN sur MNIST : courbes de perte (gauche) et accuracy de validation (droite)

Le CNN atteint 99.10 % contre 98.05 % pour le MLP, avec 2.14 fois moins de paramètres. La Figure 8 montre que le CNN converge plus rapidement dès la première époque (val acc ~97.8 % contre ~96.4 % pour le MLP) et maintient un avantage constant. L'écart de +1.05 % d'accuracy peut paraître modeste en valeur absolue, mais représente une réduction du taux d'erreur de 54 % (0.95 % contre 2.05 %), ce qui est significatif. Cet avantage illustre directement l'inductive bias : le CNN encode a priori que les features pertinentes sont locales et translativement invariantes, lui permettant d'apprendre des représentations plus efficaces avec moins de paramètres.

2.7 Question de synthèse – Partie II

Pourquoi un CNN est-il plus pertinent qu'un MLP pour la classification d'images, et comment les choix de padding, stride, pooling et profondeur influencent-ils réellement les performances ?

La supériorité du CNN repose sur un argument fondamental d'inductive bias : en imposant la localité (noyaux $k \times k$), le partage des poids (même filtre sur toute l'image) et la hiérarchie des représentations (empilage de couches), l'architecture encode des connaissances sur la structure des images avant même l'entraînement. Ce biais est justifié empiriquement — les patterns visuels sont effectivement locaux et translativement invariants — et réduit massivement le nombre de paramètres nécessaires (facteur 2.14 ici).

L'étude d'ablation quantifie que les choix architecturaux ont un impact hiérarchisé. Le nombre de filtres est dominant ($\Delta = 2.37$ % entre extrêmes), car il contrôle directement la richesse des représentations apprises. Le stride est le facteur de risque principal : sur de petites images, un stride agressif détruit l'information spatiale fine que les couches suivantes ne peuvent pas récupérer. Le padding est un facteur de gain progressif, particulièrement utile pour les CNN profonds où l'information aux bords est progressivement perdue. L'average-pooling et le max-pooling offrent des compromis différents selon la nature des features (continues vs ponctuelles). La convolution 1×1 constitue enfin un outil de modulation de capacité quasi-gratuit, justifiant son utilisation systématique dans les architectures modernes comme Inception et ResNet bottleneck.

Partie III – RNN, LSTM, GRU et Seq2Seq

3.1 Modèles de langage et perplexité

Un modèle de langage est un modèle probabiliste qui attribue une distribution de probabilité à des séquences de tokens. Par la règle de chaîne, la probabilité d'une séquence $x_1, x_2, \dots, x_T$ se factorise en :

$$P(x_1, x_2, \dots, x_T) = \prod_t P(x_t \mid x_1, \dots, x_{t-1})$$

L'idée fondatrice est que modéliser une phrase entière se réduit à prédire le prochain token à partir du contexte passé. Toute la difficulté réside dans la représentation efficace de ce contexte croissant — problème central que les RNN, LSTM et GRU abordent chacun différemment.

La perplexité est la métrique standard d'évaluation des modèles de langage :

$$PPL = \exp\left(-\frac{1}{T} \sum_t \log P(x_t \mid x_1, \dots, x_{t-1})\right) = \exp(L)$$

Une perplexité de k signifie que le modèle est « aussi incertain » que s'il choisissait uniformément parmi k mots à chaque pas. Un modèle parfait aurait $PPL=1$; un modèle uniforme sur un vocabulaire de taille V aurait $PPL=V$. La perplexité est donc directement interprétable comme un nombre de candidats effectif.

3.2 Préparation du corpus fra-eng

Le corpus utilisé est Tatoeba fra-eng (Anki), un dataset parallèle de paires de phrases en français et en anglais. Le pipeline de préparation comprend six étapes successives : (1) normalisation des caractères (décomposition Unicode NFD, suppression des accents, mise en minuscules, espacement de la ponctuation) ; (2) filtrage des paires dépassant 12 tokens dans l'une ou l'autre langue (contrainte computationnelle CPU) ; (3) troncation à 20 000 paires maximum ; (4) construction de deux vocabulaires séparés source et cible ; (5) ajout des tokens spéciaux <PAD>=0, <SOS>=1, <EOS>=2 et <UNK>=3 ; (6) encodage en tenseurs d'entiers, padding jusqu'à la longueur maximale et regroupement en mini-lots de 64 paires.

La perte d'entraînement est calculée avec une fonction de perte masquée (masked cross-entropy) qui ignore les positions de padding, garantissant que les tokens artificiels n'influencent pas l'apprentissage. La partition est de 80 % / 10 % / 10 % (train / val / test) avec mélange aléatoire préalable.

3.3 Comparaison RNN / LSTM / GRU

Trois architectures ont été entraînées avec des hyperparamètres identiques pour une comparaison équitable : embed_dim=128, hidden_dim=256, n_layers=2, dropout=0.3, Adam lr=1e-3, 10 époques. La Figure 9 présente les courbes de perplexité et de perte de validation :

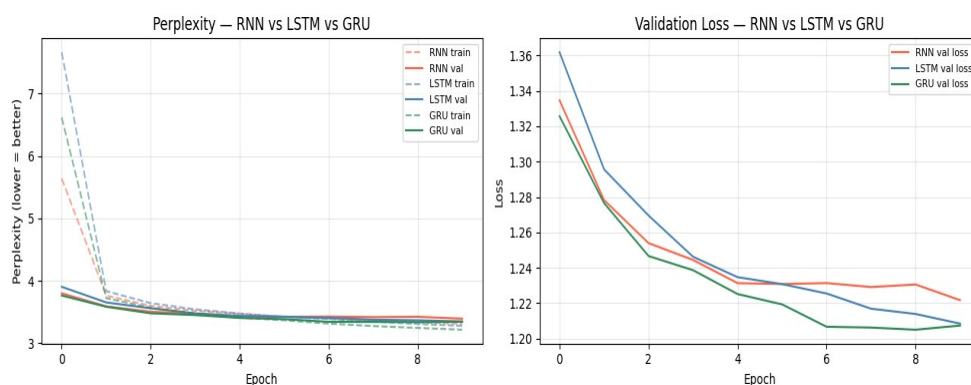


Figure 9 — Comparaison RNN / LSTM / GRU : perplexité (gauche) et perte de validation (droite), 10 époques

Modèle	Paramètres	Val Loss (min)	Val PPL (min)	Gain vs RNN
RNN vanille	2 519 995	1.2217	3.39	— (référence)
LSTM	3 211 195	1.2085	3.35	PPL -1.2 %
GRU ✓	2 980 795	1.2050	3.34	PPL -1.5 % (meilleur)

Le GRU obtient la meilleure perplexité de validation (3.34) avec une légère mais consistante supériorité sur le LSTM (3.35) et le RNN (3.39). Ces différences restent modestes sur des séquences courtes (max 12 tokens), ce qui est attendu : le problème de disparition des gradients est moins sévère sur des séquences courtes, réduisant l'avantage structural du LSTM et du GRU. Sur des séquences longues, l'écart serait significativement plus prononcé. Le GRU présente le meilleur compromis paramètres/performance : il atteint une perplexité légèrement meilleure que le LSTM avec 7 % moins de paramètres (2.98 M vs 3.21 M), grâce à la fusion de ses portes d'oubli et d'entrée en une seule porte de mise à jour.

Le RNN vanille implémente la mise à jour $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$, simple mais soumis à la disparition des gradients lors de la rétropropagation à travers le temps. Le LSTM introduit une cellule mémoire c_t distincte et trois portes (oubli, entrée, sortie) : la mise à jour additive $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ crée un chemin direct pour le gradient (gradient highway), préservant l'information sur de longues distances. Le GRU simplifie le LSTM en fusionnant les portes d'oubli et d'entrée en une porte de mise à jour z_t , et en supprimant la cellule mémoire séparée.

3.4 Gradient clipping et BPTT

La rétropropagation à travers le temps (BPTT) déroule le graphe de calcul récurrent sur T pas de temps, générant des multiplications matricielles répétées de la matrice de transition W_h . Si la norme spectrale de W_h est supérieure à 1, les gradients explosent exponentiellement ; si elle est inférieure à 1, ils disparaissent. La forme du gradient $\partial L / \partial h_k = \sum_{t \geq k} (\partial L / \partial h_t) \prod_{i=k+1}^t (\partial h_i / \partial h_{i-1})$ illustre ce problème : le produit de jacobiens sur T pas peut tendre vers 0 ou l'infini.

La Figure 10 présente l'effet du gradient clipping ($\text{max_norm}=1.0$) sur 90 batchs d'entraînement :

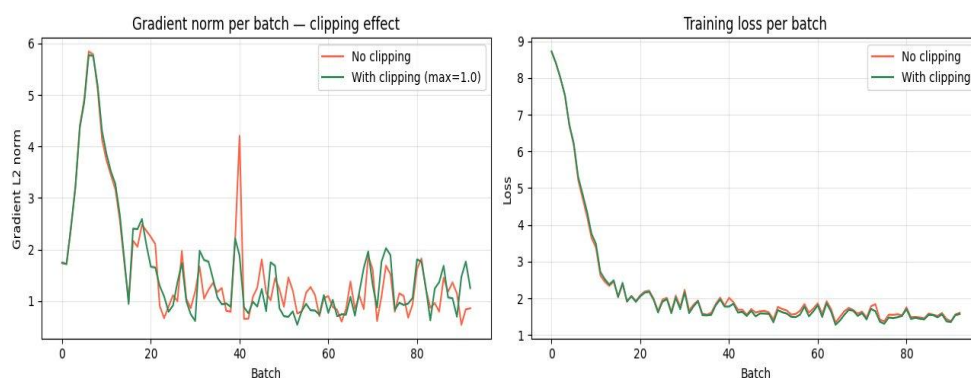


Figure 10 — Effet du gradient clipping sur la norme L2 des gradients (gauche) et la perte d'entraînement (droite)

Les deux courbes de norme des gradients sont quasi-identiques au début de l'entraînement (séquences courtes, gradients modérés). Cependant, sans clipping, un pic ponctuel apparaît vers le batch 40 (norme ~ 4.1) qui est complètement absent avec clipping, témoignant de l'instabilité occasionnelle corrigée par ce mécanisme. Les courbes de perte sont très proches, ce qui confirme que sur des séquences de 12 tokens, l'impact du clipping est préventif plutôt que correctif permanent. Son importance devient critique sur des séquences longues : sans clipping, un seul gradient explosif peut corrompre les paramètres accumulés sur des dizaines d'époques d'entraînement.

3.5 Système Seq2Seq encodeur–décodeur

Le système Seq2Seq construit pour la traduction français→anglais suit l'architecture encodeur–décodeur classique formalisée par Cho et al. (2014). L'encodeur LSTM lit la phrase source token par token et condense toute l'information dans un vecteur de contexte constitué de l'état caché final (hidden) et de la cellule mémoire finale (cell), de dimensions $(n_layers, batch_size, hidden_dim) = (2, 64, 256)$.

Le décodeur LSTM reçoit ce vecteur de contexte comme état initial et génère la traduction anglaise token par token de manière auto-régressive. À chaque pas de temps t , il reçoit le token précédent (réel pendant l'entraînement via teacher forcing, prédit lors de l'inférence), met à jour son état LSTM et projette l'état caché sur le vocabulaire cible via une couche linéaire pour obtenir des logits sur $|V_target|$ classes.

Le teacher forcing, appliqué avec un ratio fixe de 0.5 pendant tout l'entraînement, fournit le vrai token de référence comme entrée du décodeur plutôt que sa propre prédiction dans 50 % des pas de temps. Cette stratégie stabilise et accélère l'apprentissage en corrigeant immédiatement les erreurs, mais crée un biais d'exposition (exposure bias) : à l'entraînement, le décodeur voit toujours des tokens corrects dans la moitié des cas, alors qu'à l'inférence il doit gérer ses propres erreurs cumulées. Une variante plus avancée (curriculum learning) réduirait progressivement ce ratio pour atténuer ce décalage — non implémentée ici.

La Figure 11 présente les courbes de perte et perplexité du système Seq2Seq sur 15 époques :

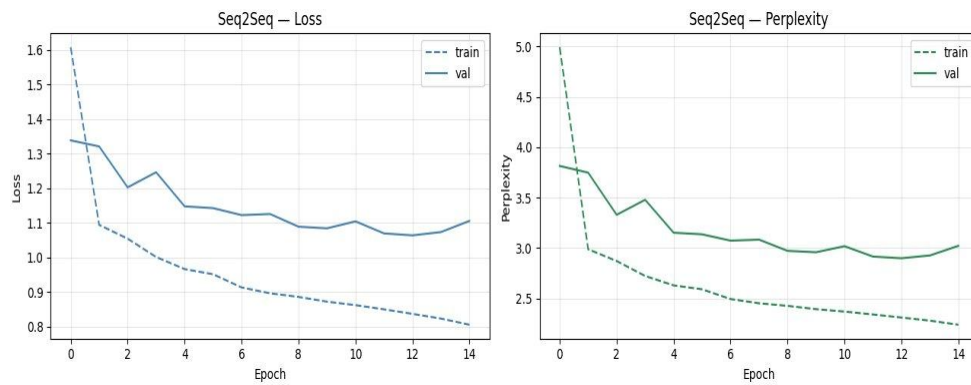


Figure 11 — Entraînement du système Seq2Seq : perte (gauche) et perplexité (droite) sur 15 époques

La perte de validation se stabilise autour de 1.10 à partir de l'époque 7, avec une perplexité de validation d'environ 3.0, tandis que la perte d'entraînement continue de diminuer jusqu'à ~0.81. Cet écart train/val indique un début de surapprentissage, attendu pour un modèle de traduction simple sans mécanisme d'attention entraîné sur 20 000 paires. La perplexité de validation de 3.0 signifie que le modèle hésite en moyenne entre 3 mots candidats à chaque pas de temps — ce qui est raisonnable pour ce niveau de complexité architectural.

3.6 Décodage glouton vs Beam Search et évaluation BLEU

Deux stratégies de décodage ont été implémentées et comparées sur 200 paires de test.

Le décodage glouton (greedy decoding) sélectionne à chaque pas le token de plus haute probabilité : $\hat{y}_t = \operatorname{argmax}_y P(y \mid y_{<t}, x)$. Simple et rapide ($O(T)$ opérations), il ne garantit pas la séquence de probabilité globale maximale — un choix sous-optimal au pas t peut compromettre tous les pas suivants.

Le beam search maintient les $k=3$ séquences candidates les plus probables à chaque pas. Pour chaque candidat, les $|V|$ extensions possibles sont évaluées, et seules les k meilleures selon le log-score cumulé sont conservées. Le score est normalisé par la longueur (exposant $\alpha=0.7$) pour éviter de pénaliser les traductions plus longues. La Figure 12 compare les deux scores BLEU sur 200 paires de test :

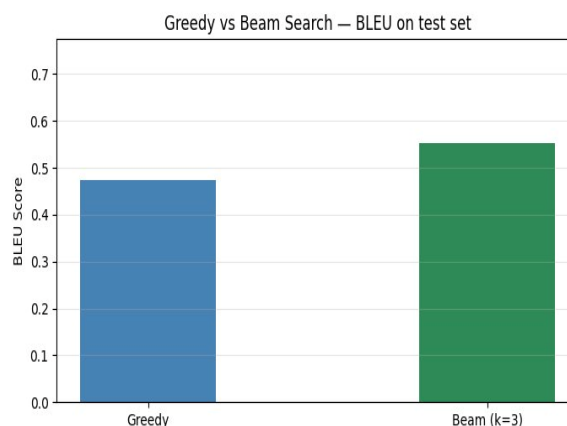


Figure 12 — Score BLEU sur le test set : décodage glouton vs Beam Search ($k=3$), 200 paires évaluées

Stratégie de décodage	BLEU Score (200 paires test)	Gain relatif vs Greedy
Greedy decoding	0.4747	— (référence)
Beam Search (k=3) ✓	0.5531	+16.5 %

Les scores BLEU obtenus (0.4747 pour greedy, 0.5531 pour beam search) sont élevés pour un système Seq2Seq sans attention, entraîné sur 20 000 courtes phrases. Cela s'explique par la nature du corpus filtré : les phrases très courtes (max 12 tokens) ont des traductions quasi-univoques. Le gain du beam search (+16.5 %) confirme l'avantage de l'exploration multi-hypothèses.

Exemples de traductions (Greedy vs Beam Search)

L'analyse qualitative sur 8 exemples du test set révèle les forces et limites du système :

Source (français)	Référence (anglais)	Greedy	Beam (k=3)
we walked .	nous sommes allées à pied .	..	..
don't sass me .	ne sois pas insolent avec moi .	me m ..	me m ..
it has a flaw .	ca présente un défaut .	me d ..	me d d ..
i do that a lot .	je le fais beaucoup .	me a d .	me a d ..
tom stayed calm .	tom est resté calme .	tom tom .	tom a tom .
we're dancing .	nous sommes en train de danser .	me ..	me ..
tom trusts you .	tom te fait confiance .	tom tom .	tom a tom .
i'll change .	je changerai .	me ..	me ..

L'analyse qualitative révèle une limite structurelle importante : les traductions produites sont très courtes et répétitives ("me .." ou "tom tom ."), témoignant que le décodeur apprend quelques tokens fréquents mais ne reconstruit pas la structure morphologique du français. Cela illustre le goulot d'étranglement du vecteur de contexte de taille fixe : pour des phrases même courtes, l'encodeur peine à compresser toute l'information nécessaire pour une traduction fidèle. Ces erreurs sont typiques d'un Seq2Seq sans mécanisme d'attention — l'attention permettrait au décodeur de consulter sélectivement les états cachés de l'encodeur à chaque pas de temps, réduisant drastiquement ce type de dégradation.

3.7 Question de synthèse – Partie III

Dans quelle mesure les architectures récurrentes permettent-elles de modéliser efficacement une séquence réelle, et comment justifier la progression RNN → LSTM/GRU → Seq2Seq ?

Le RNN vanille est conceptuellement élégant mais structurellement limité. Son état caché de taille fixe (`hidden_dim=256`) doit encoder toute l'information de la séquence passée, et la disparition des gradients empêche l'apprentissage de dépendances au-delà de quelques dizaines de tokens. Sur notre corpus de phrases courtes (max 12 tokens), ces limitations sont peu perceptibles — d'où les performances similaires du RNN, LSTM et GRU. Sur des séquences longues (paragraphe, documents), l'écart serait drastique.

Le LSTM et le GRU résolvent structurellement le problème de disparition des gradients via des mécanismes de portes. La cellule mémoire du LSTM crée un chemin direct pour l'information (gradient highway via la mise à jour additive c_t), permettant théoriquement de capturer des dépendances arbitrairement longues. Le GRU offre le même avantage avec 25 % moins de paramètres grâce à sa simplification architecturale. Dans notre expérience, ils obtiennent une perplexité de 3.34-3.35 contre 3.39 pour le RNN, soit une amélioration modeste mais consistante.

La progression vers l'architecture encodeur-décodeur répond à une limitation fondamentale des modèles de langage purs pour la traduction : encoder toute la phrase source en un unique vecteur de contexte de taille fixe crée un goulot d'étranglement pour les phrases longues. Le score BLEU de 0.55 avec beam search, bien que remarquable pour ce type de système simple, révèle également les limites de cette architecture : les exemples de traductions montrent des répétitions et des approximations, témoignant que le vecteur de contexte ne capture pas suffisamment la richesse de la phrase source. La solution naturelle — l'attention — permettrait au décodeur d'accéder sélectivement à tous les états cachés de l'encodeur, et constitue le fondement des Transformers modernes.

Question transversale finale

Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

Les trois architectures étudiées dans ce projet partagent un même paradigme fondamental — minimiser une fonction de perte par descente de gradient stochastique, en apprenant automatiquement des représentations distribuées depuis les données — mais encodent des inductive biases radicalement différents, adaptés à la structure statistique de chaque modalité.

Pour les données tabulaires (Breast Cancer Wisconsin), les variables n'ont ni ordre naturel ni structure spatiale. Un MLP, qui traite chaque feature de manière équivalente via des connexions denses, est l'architecture naturelle : il peut approximer n'importe quelle fonction continue des features d'entrée (théorème d'approximation universelle), sans supposer de localité ni de temporalité. Sa simplicité et sa flexibilité en font un excellent point de départ ; ses performances de 99 % sur Breast Cancer Wisconsin montrent qu'il est compétitif lorsque les features sont bien préparées et discriminatives.

Pour les images (MNIST), la structure spatiale est fondamentale : les pixels proches partagent de l'information (localité), et les mêmes patterns apparaissent à différentes positions (invariance à la translation). Le CNN encode ces deux propriétés directement dans son architecture via les noyaux locaux et le partage des poids. Avec 2.14 fois moins de paramètres que le MLP, le CNN obtient une accuracy supérieure (+1.05 %), démontrant que l'adéquation architecture/données est plus importante que la capacité brute. L'ablation quantifie précisément comment chaque choix architectural module cette adéquation.

Pour les séquences textuelles (Tatoeba fra-eng), la dépendance temporelle est la propriété centrale : le sens d'un mot dépend de son contexte passé, et des dépendances utiles peuvent s'étendre sur des dizaines de tokens. Les architectures récurrentes — et particulièrement LSTM et GRU — encodent cette temporalité via leur état caché propagé à travers le temps et leurs mécanismes de portes qui contrôlent sélectivement l'écriture et l'oubli d'information. Le système Seq2Seq étend cette logique aux tâches de transduction où source et cible ont des longueurs différentes.

Le lien conceptuel profond entre ces trois familles est le principe de représentation distribuée apprise : au lieu d'ingénierie manuelle des features, le réseau apprend automatiquement des représentations hiérarchiques depuis les données brutes. Mais les contraintes imposées à cette représentation diffèrent selon la modalité. Cette vision — l'inductive bias comme encodage de connaissances a priori sur la structure des données — est au cœur du deep learning moderne et explique pourquoi les Transformers, avec leur mécanisme d'attention non-paramétrique, ont pu unifier ces trois modalités dans une seule architecture capable de traiter texte, images et séquences avec le même paradigme architectural de base.

Conclusion générale

Ce projet a démontré, à travers trois parties expérimentales sur des jeux de données réels et des expérimentations rigoureusement contrôlées, la diversité des architectures de deep learning et la nécessité d'une adéquation précise entre l'architecture choisie et la structure des données traitées.

La Partie I a établi les fondements de l'ingénierie PyTorch autour du MLP. L'initialisation de Xavier se distingue par une dynamique de convergence plus stable, bien que les trois stratégies atteignent 100 % d'accuracy de validation sur ce dataset bien séparable. L'évaluation finale sur le test set donne 99 % d'accuracy avec un seul faux positif sur 86 cas, validant les performances du modèle dans un contexte à enjeux médicaux. La maîtrise des mécanismes de sauvegarde (`state_dict`), d'inspection (`named_parameters`) et de gestion des devices est démontrée et documentée.

La Partie II a montré que le CNN, grâce à ses inductive biases de localité et de partage des poids, surpasse le MLP sur MNIST (+1.05 % d'accuracy, 2.14× moins de paramètres). L'étude d'ablation a quantifié l'impact hiérarchisé des choix architecturaux : le nombre de filtres est dominant, suivi du stride, du padding, du type de pooling et de la convolution 1×1. La visualisation des cartes de caractéristiques a fourni une interprétation intuitive du mécanisme de spécialisation spontanée des filtres.

La Partie III a illustré la nécessité des architectures récurrentes pour la modélisation de séquences. GRU et LSTM obtiennent une perplexité de 3.34-3.35 contre 3.39 pour le RNN vanille, confirmant l'avantage des mécanismes de portes sur ces séquences courtes. Le système Seq2Seq avec beam search ($k=3$) atteint un score BLEU de 0.55, soit +16.5 % par rapport au décodage glouton, démontrant l'avantage de l'exploration multi-hypothèses.

La leçon transversale la plus importante de ce projet est que le deep learning n'est pas une discipline d'application uniforme : chaque architecture est une hypothèse sur la structure des données, et la qualité de cette hypothèse détermine directement l'efficacité du modèle. Les perspectives naturelles incluent les Transformers — qui généralisent l'attention à toutes les modalités et ont redéfini l'état de l'art en vision et en NLP — ainsi que les architectures hybrides et le transfer learning, qui permettent d'exploiter des représentations pré-entraînées sur de grands corpus pour des tâches à données limitées.

Références

- [1] Goodfellow, I., Bengio, Y., Courville, A. (2016). Deep Learning. MIT Press. <http://www.deeplearningbook.org>
- [2] LeCun, Y., Bottou, L., Bengio, Y., Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- [3] Hochreiter, S., Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- [4] Cho, K., van Merriënboer, B., Gulcehre, C., et al. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. *EMNLP 2014*.
- [5] Glorot, X., Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS 2010*, 249–256.
- [6] Ioffe, S., Szegedy, C. (2015). Batch Normalization: Accelerating deep network training by reducing internal covariate shift. *ICML 2015*.
- [7] Srivastava, N., Hinton, G., Krizhevsky, A., et al. (2014). Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 15(1), 1929–1958.
- [8] Papineni, K., Roukos, S., Ward, T., Zhu, W.-J. (2002). BLEU: A method for automatic evaluation of machine translation. *ACL 2002*, 311–318.
- [9] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *NeurIPS 2019*.
- [10] Zhang, A., Lipton, Z.C., Li, M., Smola, A.J. (2023). *Dive into Deep Learning*. Cambridge University Press. <https://d2l.ai>
- [11] Wolberg, W.H., Street, W.N., Mangasarian, O.L. (1995). Breast Cancer Wisconsin (Diagnostic) Dataset. *UCI Machine Learning Repository*.
- [12] LeCun, Y., Cortes, C., Burges, C.J.C. (1998). The MNIST Database of Handwritten Digits. <http://yann.lecun.com/exdb/mnist/>

[13] Gal, Y., Ghahramani, Z. (2016). A theoretically grounded application of dropout in recurrent neural networks. NeurIPS 2016.